Web Foundations: Responsive DOM Manipulation & CSS Grid

Semantic HTML, DOM events, accessibility, responsive layout, and robust browser behavior

1. Semantic HTML as the foundation

HTML defines document structure and meaning; CSS controls presentation; JavaScript adds behavior. Use semantic elements such as main, nav, section, article, button, and form according to their purpose. Semantic markup supports keyboard interaction, accessibility APIs, browser defaults, and maintainability.

```
<main>
  <h1>Tasks</h1>
  <form id="task-form">
    <label for="task-name">New task</label>
    <input id="task-name" name="task" required maxlength="120" />
    <button type="submit">Add task</button>
  </form>
  <ul id="task-list" aria-live="polite"></ul>
</main>
```

Design notes: Use buttons for actions and anchors for navigation. Associate labels with inputs, keep heading hierarchy meaningful, and avoid choosing elements only for their default appearance. Include a viewport meta tag for responsive layouts.

Practice: Inspect the page with keyboard navigation and a screen reader outline; verify that labels and actions have clear names.

2. Selecting and updating DOM elements

The DOM represents the page as objects that scripts can query and modify. Use querySelector/querySelectorAll for CSS-selector-based selection and textContent for plain text. Keep selectors specific enough to be stable without depending on fragile presentation classes.

```
const title = document.querySelector('#page-title');
if (title) {
  title.textContent = 'Updated title';
  title.classList.add('is-ready');
}
```

Design notes: An element lookup can return null; guard it when scripts may run on pages without that element. Prefer textContent for untrusted text rather than innerHTML, which parses markup and can create injection risks.

Practice: Write a function that updates a status message and handles the case where the target element is missing.

Additional review: Implementation lens: isolate the smallest reproducible example, state the expected result, and test it independently before integrating it into a larger application.

3. Events and event delegation

Event listeners connect user actions to behavior. Event bubbling allows an event from a child element to propagate to ancestors. Delegation attaches one listener to a stable parent and checks the event target, which is useful for lists whose items are added dynamically.

```
const list = document.querySelector('#task-list');

list?.addEventListener('click', (event) => {
  const button = event.target.closest('button[data-action="remove"]');
  if (!button || !list.contains(button)) return;
  button.closest('li')?.remove();
});
```

Design notes: Use event.currentTarget when you need the element that owns the listener, and event.target for the origin of the event. Be careful with nested controls and propagation. Avoid attaching duplicate listeners each time a component re-renders.

Practice: Add and remove list items dynamically while keeping only one click listener on the list.

4. Safe dynamic content and DOM injection

Untrusted strings should be treated as text, not HTML. Assigning innerHTML can create cross-site scripting vulnerabilities when strings contain attacker-controlled markup. Build elements with createElement and set textContent for plain text content.

```
function addTask(list, label) {
  const item = document.createElement('li');
  const text = document.createElement('span');
  const remove = document.createElement('button');

  text.textContent = label;
  remove.type = 'button';
  remove.dataset.action = 'remove';
  remove.textContent = 'Remove';
  item.append(text, ' ', remove);
  list.append(item);
}
```

Design notes: Setting textContent avoids parsing a string as HTML, but URL attributes, CSS, and other contexts have their own security rules. Validate data by context and do not rely on client-side validation as a security boundary.

Practice: Enter a string containing HTML tags into the task form and verify it displays literally rather than creating elements.

5. Forms, validation, and state synchronization

The browser provides constraint validation for required fields, input types, and length/range constraints. JavaScript can enhance the experience, but should preserve meaningful labels, visible errors, and keyboard access. DOM state and application state should not silently drift apart.

```
const form = document.querySelector('#task-form');
const input = document.querySelector('#task-name');
const list = document.querySelector('#task-list');

form?.addEventListener('submit', (event) => {
  event.preventDefault();
  const value = input.value.trim();
  if (!value) {
    input.setCustomValidity('Enter a task name.');
    input.reportValidity();
    return;
  }
  input.setCustomValidity('');
  addTask(list, value);
  form.reset();
});
```

Design notes: This snippet assumes the elements exist and that addTask is defined. Clear custom validity when the user edits the input. If data is submitted to a server, validate it again server-side.

Practice: Add an inline error summary and ensure errors are announced and associated with the relevant field.

Additional review: Review lens: consider edge cases, failure behavior, maintainability, and the assumptions the code makes about its input or execution environment.

6. CSS Grid for two-dimensional layout

CSS Grid defines rows and columns and is useful for card galleries, dashboards, and page shells. A grid container controls track sizes and gaps; grid items can span tracks. Use Grid for two-dimensional structure and Flexbox for one-dimensional alignment within components.

```
.card-grid {
  display: grid;
  grid-template-columns: repeat(1, minmax(0, 1fr));
  gap: 1rem;
}

@media (min-width: 42rem) {
  .card-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}

@media (min-width: 64rem) {
  .card-grid {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }
}
```

Design notes: minmax(0, 1fr) helps prevent long content from forcing a track wider than expected. CSS Grid placement should not undermine logical DOM order. Check for overflow with long words, large controls, and translated text.

Practice: Create a three-column desktop card grid that gracefully collapses to two and then one column.

7. Responsive design and content-first breakpoints

Responsive design adapts layout to available space, content, input mode, and user preferences. Media queries can respond to viewport width, reduced motion, color scheme, and other media features. Choose breakpoints where the content needs them rather than targeting named devices.

```
<meta name="viewport" content="width=device-width, initial-scale=1">

<style>
  .layout { display: grid; gap: 1rem; }
  @media (min-width: 48rem) {
    .layout { grid-template-columns: 16rem minmax(0, 1fr); }
  }
  @media (prefers-reduced-motion: reduce) {
    *, *::before, *::after { scroll-behavior: auto !important; }
  }
</style>
```

Design notes: Responsive behavior includes readable type, usable controls, non-overflowing tables, and images that scale. Test at narrow widths, zoomed views, and with long localized content. Do not use device detection when a layout constraint solves the problem.

Practice: Test the page at several widths and at 200% zoom; document any horizontal scrolling and whether it is intentional.

8. Accessible interactive components

Native HTML controls offer built-in keyboard behavior and semantics. Prefer native buttons, checkboxes, select elements, and dialogs where suitable. Custom controls require carefully implemented keyboard support, focus management, accessible names, and state announcements.

```
<button id="toggle-details"
        aria-expanded="false"
        aria-controls="details">
  Show details
</button>
<section id="details" hidden>
  Additional information goes here.
</section>

<script>
const toggle = document.querySelector('#toggle-details');
const details = document.querySelector('#details');
toggle?.addEventListener('click', () => {
  const expanded = toggle.getAttribute('aria-expanded') === 'true';
  toggle.setAttribute('aria-expanded', String(!expanded));
  details.hidden = expanded;
  toggle.textContent = expanded ? 'Show details' : 'Hide details';
});
</script>
```

Design notes: ARIA attributes should reflect actual state and do not replace native semantics. If a component changes focus or opens a modal, define focus placement, escape behavior, and restoration. Keep keyboard and pointer experiences equivalent.

Practice: Use Tab, Shift+Tab, Enter, Space, and Escape to test interactive elements and document expected behavior.

Additional review: Optimization lens: measure behavior with representative data and workload; check the relevant official documentation before relying on implementation-specific behavior.

9. DOM performance and maintainability

Batch DOM updates when possible and avoid repeated full-list rebuilds for every small change. Document fragments can help assemble groups of nodes before insertion, though modern browsers optimize many common operations. Measure before adding complexity.

```
const fragment = document.createDocumentFragment();
for (const label of ['Plan', 'Build', 'Test']) {
  const item = document.createElement('li');
  item.textContent = label;
  fragment.append(item);
}
document.querySelector('#task-list')?.append(fragment);
```

Design notes: Use event delegation for repeated dynamic items, clean up listeners when components are removed, and avoid layout thrashing caused by interleaving layout reads and writes in tight loops. Keep DOM state updates predictable.

Practice: Render 1,000 list items using direct appends and a fragment, then profile before drawing performance conclusions.

10. Integrated mini-project and debugging

A robust responsive interface combines semantic structure, constrained DOM manipulation, accessible events, and layout that adapts to content. Debug behavior with browser devtools: inspect the DOM, event listeners, computed styles, console errors, and network requests.

```
/* Example CSS for a resilient page shell */
.page {
  width: min(100% - 2rem, 72rem);
  margin-inline: auto;
}
.dashboard {
  display: grid;
  grid-template-columns: minmax(0, 1fr);
  gap: 1rem;
}
.panel { min-width: 0; }
@media (min-width: 56rem) {
  .dashboard { grid-template-columns: 16rem minmax(0, 1fr); }
}
```

Design notes: Keep CSS layout rules separate from behavior where possible. Test empty states, long content, failed form validation, keyboard operation, reduced-motion preferences, and narrow viewports. Do not depend on a single browser or screen size.

Practice: Capstone: build a responsive task dashboard with add/remove behavior, delegated events, validation, accessible announcements, and a Grid-based layout.

Trusted documentation and further reading

- **MDN — Document Object Model** — https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- **MDN — Introduction to events** — https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Events
- **MDN — Event bubbling** — https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Event_bubbling
- **MDN — CSS Grid Layout** — https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout
- **MDN — Responsive design** — https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design
- **MDN — Using media queries** — https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_media_queries/Using_media_queries
- **WHATWG — HTML Living Standard** — https://html.spec.whatwg.org/
- **W3C WAI — Forms concepts** — https://www.w3.org/WAI/tutorials/forms/

Subject review checklist

- Semantic HTML and form labels are present.
- Untrusted text is inserted without unsafe HTML parsing.
- Dynamic events are handled correctly.
- Responsive layouts remain usable under zoom and narrow widths.
- Keyboard focus and status announcements are tested.

Cross-topic implementation checklist

• Pin language/runtime/framework/database versions for reproducible builds.

• Prefer official versioned documentation when behavior differs between releases.

• Keep examples small enough to test independently, then integrate them into realistic projects.

• Include invalid-input tests, boundary cases, failure handling, and accessibility or security review where relevant.

• Measure actual performance and resource use rather than relying on assumptions.

• Document ownership, API contracts, schema decisions, responsive breakpoints, and operational expectations.

Source and attribution note

This PDF is an original educational synthesis. Short code examples and explanations were written for this guide; they are not copied wholesale from the linked documentation. Official documentation is linked in each topic so readers can confirm current details, exact semantics, and version-specific constraints. The C17 chapter uses the publicly available C11 committee draft as a close language reference alongside compiler tooling and secure-coding guidance; consult the applicable C17 standard where strict conformance is required.